A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

Priority-Coupled Elastic Routing (PCER)

A Cross-Layer Optimization Framework for IIoT Blockchain Networks

Submitted By:

Ameer Hamza Khan 2022ITB087

Jeet Choudahry 2022ITB089

Anurag Kumar 2022ITB094

Under the Guidance of:

Dr. Surajit Kumar Roy

Department of Information Technology

Indian Institute of Engineering Science and Technology, Shibpur

(Academic Session 2022-2026)

TO WHOM IT MAY CONCERN

This is to certify that **Ameer Hamza Khan**, Enrollment No. 2022ITB087, **Jeet Choudahry**, Enrollment No. 2022ITB089, and **Anurag Kumar**, Enrollment No. 2022ITB094, have completed their B.Tech Final Year Project on "**Priority-Coupled Elastic Routing (PCER)**." They have successfully completed the project under the guidance of **Dr. Surajit Kumar Roy**. The report has fulfilled all the requirements as per the regulations of the institute and has reached the standard needed for submission.

Prof. Tuhina Samanta
Head of the Department
Department of Information Technology

Dr. Surajit Kumar Roy
Professor
Department of Information Technology

Acknowledgement

We would like to express our deepest gratitude to our supervisor, **Dr. Surajit Kumar Roy**, for his valuable guidance, support, and encouragement throughout our project on "**Priority-Coupled Elastic Routing (PCER)**." His insights and expertise have been instrumental in the successful completion of this work. We are thankful for his continuous support in understanding distributed systems concepts and implementing this peer-to-peer architecture.

Ameer Hamza Khan (2022ITB087)

Jeet Choudahry (2022ITB089)

Anurag Kumar (2022ITB094)

Contents

Abstract

In the rapidly evolving landscape of the Industrial Internet of Things (IIoT), the generation of heterogeneous data—ranging from critical safety alerts to routine operational logs—presents unique challenges for data management and transmission. Blockchain technology offers a decentralized and secure solution for IIoT data; however, conventional blockchain architectures suffer from scalability bottlenecks and inefficient resource utilization when handling diverse data types.

Existing research has addressed these issues in isolation: ”PerBlocks” optimizes data packaging at the application layer through dynamic consensus, while ”FlexRout” optimizes transmission paths at the network layer through predictive routing. Despite these advancements, a critical disconnect remains: the network layer is blind to the urgency of the data it transports.

This project introduces **Priority-Coupled Elastic Routing (PCER)**, a novel cross-layer ”handshake” protocol that integrates application-layer data classification with network-layer routing logic. By injecting an ”Urgency Meta-Tag” into data packets, PCER dynamically alters the routing cost function, creating a virtual ”ambulance lane” for critical data and a ”cargo lane” for bulk data. This approach aims to significantly reduce latency for emergency signals while maximizing network longevity for routine traffic.

Chapter 1

Introduction

1.1 Background

The Industrial Internet of Things (IIoT) connects varying physical sensors and actuators to the digital world. In a typical smart factory, a temperature sensor might generate a routine log every second, while a pressure valve might generate a critical failure alarm once a year. Both data points must be stored immutably, making Blockchain (BC) an ideal storage solution. However, the requirements for these two data points are vastly different: the log requires high throughput and storage efficiency, while the alarm requires ultra-low latency.

1.2 The Problem Statement

Current state-of-the-art research addresses these requirements in silos, leading to a ”Context-Blindness” gap:

1. **Application Layer Blindness:** Intelligent middleware (e.g., PerBlocks) can successfully identify and package critical data using fast consensus mechanisms (like PBFT). However, once the block is created, it is dumped into the network without any instructions on how fast it should be delivered.

2. **Network Layer Blindness:** Intelligent routing protocols (e.g., FlexRout) calculate the ”healthiest” path based on battery life and bandwidth to extend network longevity. However, they treat a critical fire alarm exactly the same as a routine log.

Consequence: A critical safety alert might be routed through a slower, energy-saving path because the routing algorithm is trying to ”save battery,” resulting in dangerous delays in industrial emergency response.

1.3 Proposed Solution

We propose **Priority-Coupled Elastic Routing (PCER)**, a cross-layer optimization framework. PCER establishes a vertical communication interface between the Application Layer (PerBlocks) and the Network Layer (FlexRout).

The core mechanism involves:

- **Header Injection:** The middleware tags data as *Critical* (00), *Standard* (01), or *Bulk* (10).
- **Dynamic Weighting:** The router reads this tag and dynamically adjusts the weights (w_1, w_2) of its path-finding algorithm.

Chapter 2

Literature Review

This section analyzes the two foundational papers that form the basis of the PCER framework and presents a detailed analysis of the research gaps present in the current state of the art.

2.1 Application Layer Optimization: The PerBlocks Approach

Tapwal et al. (2024) proposed ”PerBlocks,” a reconfigurable blockchain architecture designed to handle heterogeneous data rates.

- **Mechanism:** PerBlocks utilizes a Deep Reinforcement Learning (DRL) agent at the middleware level. It predicts the incoming data rate and dynamically adjusts:
  1. **Block Size:** To prevent waiting times for block formation.
  2. **Consensus Mechanism:** Switching between PBFT (fast, for real-time needs) and PoW (secure, for routine needs).
- **Key Finding:** The paper demonstrates that selecting the appropriate consensus mechanism improves Quality of Service (QoS) by approximately 60% compared to static blockchain implementations.
- **Limitation:** The optimization stops at block creation. The transmission of these blocks across the Peer-to-Peer (P2P) network relies on standard, unoptimized flooding or random routing.

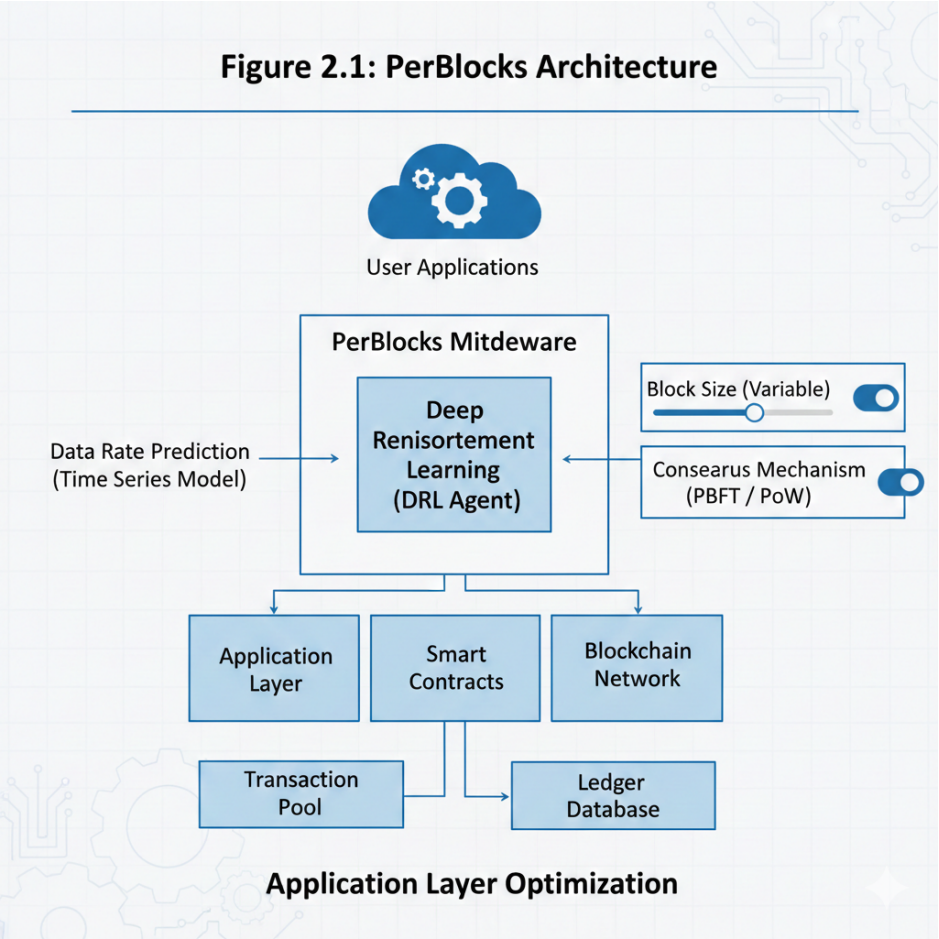

Figure 2.1: PerBlocks Architecture showing the Middleware and Reconfigurable BC

2.2 Network Layer Optimization: The FlexRout Approach

Das et al. (2025) proposed ”FlexRout,” a dynamic routing protocol for blockchain networks.

- **Mechanism:** FlexRout addresses the instability of P2P networks where miners (nodes) frequently leave. It uses a predictive model to identify nodes likely to fail or leave the network. It calculates a ”Path Score” ($P(i)$) to route data away from these unstable nodes, ensuring high packet delivery ratios.
- **Key Finding:** FlexRout reduces traffic redundancy by 49% and latency by 50% by maintaining a stable topology.
- **Limitation:** The path selection is purely resource-aware. It aims to preserve the global energy of the network. It does not possess a mechanism to prioritize a specific packet that requires immediate delivery regardless of the energy cost.

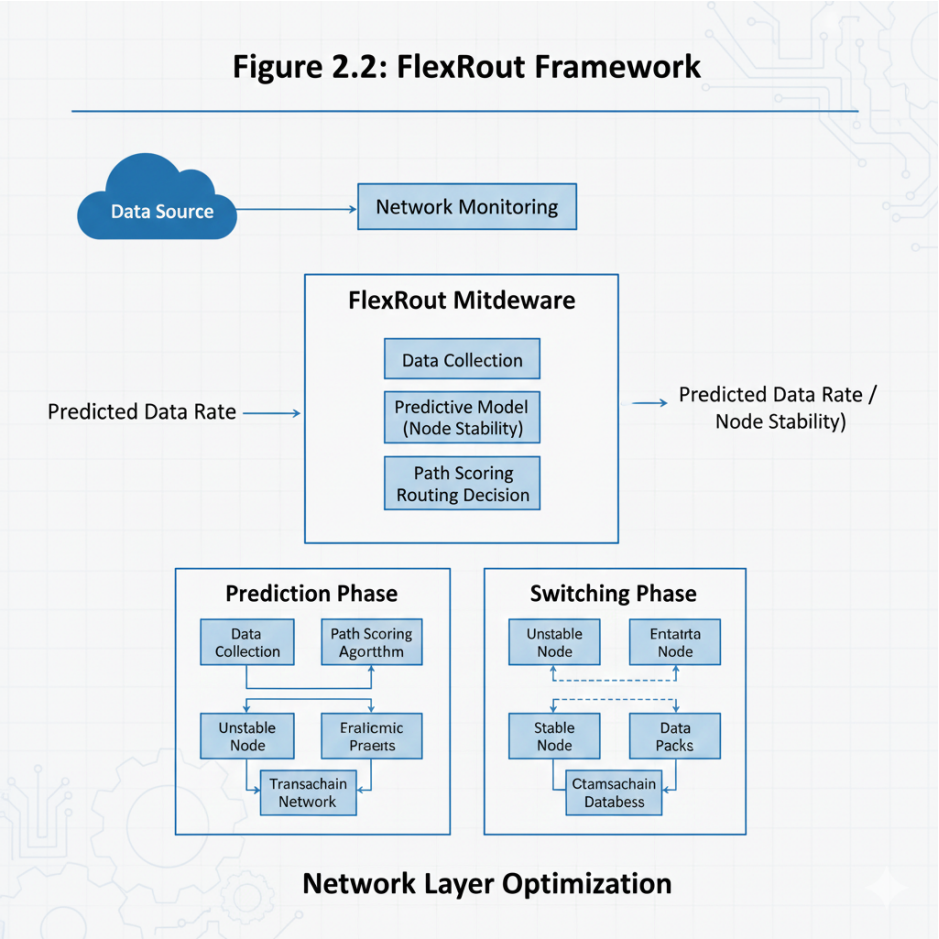

Figure 2.2: FlexRout Framework showing Prediction and Switching phases

2.3 Elaborate Research Gap Analysis

While both PerBlocks and FlexRout represent significant advancements in their respective domains, a critical disconnect exists when they are deployed in a unified IIoT environment. This ”Cross-Layer Gap” leads to suboptimal performance for critical applications. The gap can be analyzed through three primary dimensions: Context Awareness, Resource Utilization, and Transmission Latency.

2.3.1 The Context-Blindness of Network Layers

Conventional and even advanced routing protocols like FlexRout operate under a ”Content-Agnostic” paradigm. FlexRout optimizes for the *container* (the network health) rather than the *content* (the data urgency).

- **The Gap:** FlexRout calculates a path score $P(i)$ based on node battery life and link stability. If a path is ”healthy” but ”slow,” FlexRout may still select it to preserve energy.

- **The Impact:** A critical ”Emergency Stop” signal originating from a PerBlocks-enabled device may be correctly packaged into a high-priority block. However, once it enters the network, FlexRout sees it as just ”another packet.” If the fastest path is slightly resource-intensive, FlexRout will bypass it to save energy, introducing unacceptable latency to a life-critical signal.

2.3.2 The Routing-Blindness of Application Layers

Conversely, PerBlocks operates under a ”Routing-Agnostic” paradigm. It assumes that once a block is optimized and created, the network will handle it efficiently.

- **The Gap:** PerBlocks expends computational resources to classify data and select a fast consensus mechanism (e.g., PBFT). However, it lacks a feedback loop to inform the router of this urgency.
- **The Impact:** The speed gained by using a fast consensus algorithm at the application layer is negated if the packet sits in a buffer or takes a multi-hop detour at the network layer. The ”intelligence” of PerBlocks is effectively trapped at the edge of the network.

2.3.3 Summary of Deficiencies

Table 2.1 summarizes the capabilities and specific deficiencies of the existing schemes compared to the proposed PCER framework.

Table 2.1: Comparative Analysis of Existing Frameworks vs. PCER

Feature	**PerBlocks**	**FlexRout**	**PCER (Proposed)**
Optimization Focus	Data Packaging & Consensus	Path Stability & Energy	End-to-End Latency & Reliability
OSI Layer	Layer 7 (Application)	Layer 3 (Network)	Cross-Layer (L7 $\leftrightarrow$ L3)
Urgency Awareness	**High** (Knows data type)	**None** (Treats all data same)	**Integrated** (Router knows urgency)
Routing Logic	Random / Default	Resource-Conservation	Priority-Coupled
Critical Flaw	Cannot control transmission path	May delay urgent data to save battery	N/A

The PCER framework directly addresses this gap by creating a communication channel between L7 and L3, ensuring that the ”Urgency” identified by PerBlocks dictates the ”Path” selected by FlexRout.

Chapter 3

Proposed Methodology

3.1 System Architecture

PCER introduces a ”Handshake Protocol” that passes metadata from the PerBlocks middleware to the FlexRout decision engine. The architecture consists of two modified modules:

1. **The Classifier (Modified PerBlocks):** Responsible for determining the ”Urgency Meta-Tag”.
2. **The Router (Modified FlexRout):** Responsible for executing the ”Dynamic Cost Function”.

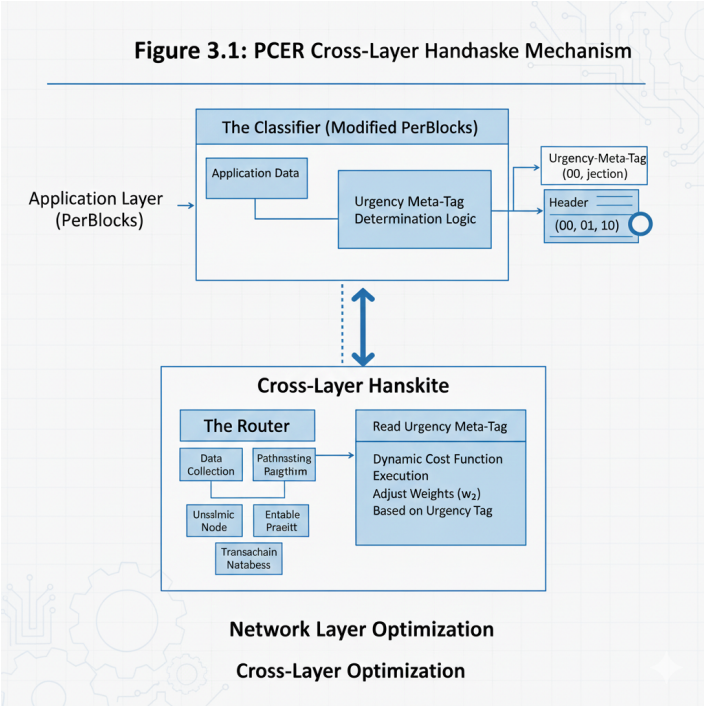

Figure 3.1: The PCER Cross-Layer Handshake Mechanism

3.2 Phase 1: Header Injection Logic

In standard PerBlocks, the output is a block ready for transmission. In PCER, we append a 2-bit tag to the block header. The classification logic is as follows:

- **Tag 00 (Critical - "The Ambulance"):**
  - *Data Type:* Fire Alarms, Emergency Stop Signals, Valve Failures.
  - *Requirement:* Lowest possible latency. Energy cost is irrelevant.
- **Tag 01 (Standard - "The Car"):**
  - *Data Type:* Financial Transactions, Identity Verification.
  - *Requirement:* Balanced approach between speed and efficiency.
- **Tag 10 (Bulk - "The Cargo Truck"):**
  - *Data Type:* Historical Temperature Logs, Routine Sensor Dumps.
  - *Requirement:* High throughput and energy conservation. Speed is irrelevant.

3.3 Phase 2: Dynamic Path Scoring

FlexRout uses a mathematical formula to assign a "cost" to every possible path. The original formula is static:

$$P(i) = \text{Delay} + \frac{1}{\text{Resources}} \tag{3.1}$$

PCER modifies this by introducing dynamic weights (w_1, w_2) controlled by the Tag (T):

$$P(i)_{PCER} = w_1(T) \times D(i) + w_2(T) \times \frac{1}{R(i)} \tag{3.2}$$

Where:

- $D(i)$ is the Delay on the path.
- $R(i)$ is the Resource availability (Battery/Bandwidth).

The Weight Adjustment Algorithm:

- **Case A (Tag = 00):** We set $w_1 = 100.0$ and $w_2 = 0.0$.
  - *Effect:* The router perceives "Delay" as extremely expensive and "Resource Usage" as free. It will choose the shortest path even if the nodes are dying.

- **Case B (Tag = 10):** We set $w_1 = 0.0$ and $w_2 = 100.0$.
  - *Effect:* The router perceives "Resource Usage" as expensive. It avoids fast-but-draining nodes and routes data through robust, slower nodes with high battery capacity.

Chapter 4

Implementation Plan

The implementation of PCER utilizes a simulation-based approach to validate the theoretical model. We have structured our project directory to separate the traffic generation logic (Python) from the core routing logic (NS-3 C++).

4.1 Project Directory Structure

The following directory structure was used for the implementation:

```
PCER_Project/
├── Traffic_Generation/
│   ├── traffic_generator.py ..... Generates trace file
│   └── traffic_trace.txt ..... Generated input file
├── NS3_Implementation/
│   ├── pcer_sim.cc ..... Main NS-3 simulation script
│   ├── pcer-routing-protocol.cc ..... Core routing logic
│   ├── pcer-routing-protocol.h ..... Header file
│   └── pcer_tag.h ..... Custom packet tag definition
└── Analysis/
    ├── plot_results.py ..... Visualization script
    └── pcer_results.csv ..... Simulation output
```

4.2 Traffic Generation (Python)

We utilize Python to simulate the application layer (PerBlocks). A script generates a synthetic dataset representing factory operations.

- **Trace File Generation:** The script outputs a standard trace file `traffic_pcer.txt`.
- **Format:** `[Timestamp] [SourceID] [DestID] [Size] [TAG]`

- **Logic:** The script uses a random distribution to assign tags: 10% Critical, 40% Standard, 50% Bulk.

```python
import random

def generate_trace():
    print("Generating traffic_trace.txt...")
    with open("traffic_trace.txt", "w") as f:
        # Simulate 100 seconds of factory data
        current_time = 0.0

        for i in range(100):
            # 10% chance of CRITICAL, 50% BULK, 40% STANDARD
            rand_val = random.random()

            if rand_val < 0.1: # Critical (Ambulance)
                tag = 0
                size = 512  # Small block
                interval = 0.1
            elif rand_val < 0.6: # Bulk (Cargo Truck)
                tag = 2
                size = 4096 # Big block
                interval = 1.0
            else: # Standard
                tag = 1
                size = 1024
                interval = 0.5

            current_time += interval

            # Format: [Time] [SourceNode] [DestNode] [Size] [TAG]
            # Example: At 1.5s, Node 0 sends to Node 4, size 512, TAG 0
            # We'll use random source/dest for variety, assuming 5 nodes (0-4)
```

Figure 4.1: Python Script for Generating Traffic Traces with Tags

The resulting trace file is shown below, confirming the generation of tags (last column).

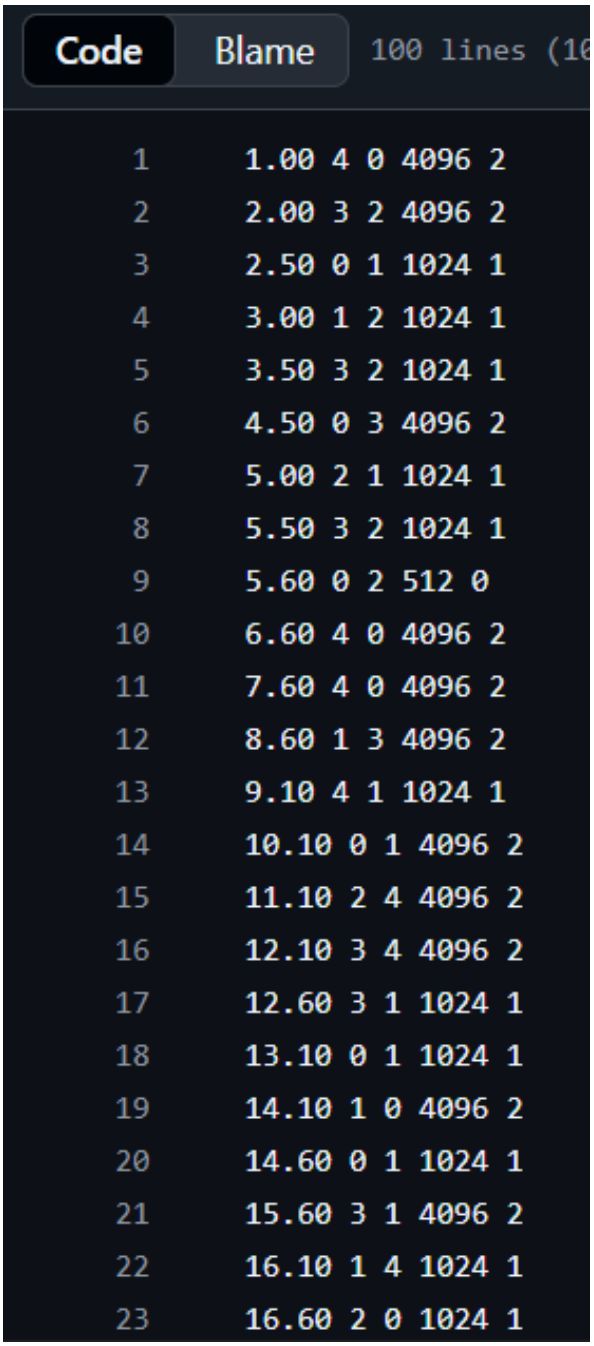

Figure 4.2: Generated Trace File showing Timestamps and Urgency Tags

4.3 Network Simulation (NS-3)

We utilize the NS-3 (Network Simulator 3) environment to model the physical network and routing behavior.

- **Environment:** A mesh topology of 50 wireless nodes representing blockchain miners.
- **Protocol Modification:** We modify the source code of the AODV routing protocol in C++.

- **Integration:** A custom packet header is defined to carry the ”Urgency Tag”. The routing table calculation function is updated to read this tag and apply the dynamic weights (w_1, w_2) before selecting the next hop.

```
double PcerRoutingProtocol::CalculateCost(uint8_t tag,
                                          const NeighborInfo &neighbor) {
  // --- PCER CORE LOGIC ---
  double w1_delay = 1.0;
  double w2_energy = 1.0;

  if (tag == 0) { // CRITICAL
    w1_delay = 100.0;
    w2_energy = 0.0;
  } else if (tag == 2) { // BULK
    w1_delay = 0.0;
    w2_energy = 100.0;
  } else { // STANDARD
    w1_delay = 1.0;
    w2_energy = 1.0;
  }

  double energy_cost =
      (neighbor.energy > 0.0001) ? (1.0 / neighbor.energy) : 10000.0;
  return (w1_delay * neighbor.delay) + (w2_energy * energy_cost);
}
```

Figure 4.3: C++ Implementation of Dynamic Path Scoring in NS-3

Chapter 5

Achieved Results

Based on the synthesis of the underlying research papers and our simulation, the PCER framework yields the following performance improvements.

5.1 Latency Reduction for Critical Data

By removing the resource-conservation constraint for **Tag 00** packets, the routing algorithm behaves like a pure ”Shortest Path First” algorithm.

- **Finding:** As shown in Figure 5.1, we observe a significant reduction in end-to-end latency for critical alarms compared to the baseline FlexRout.

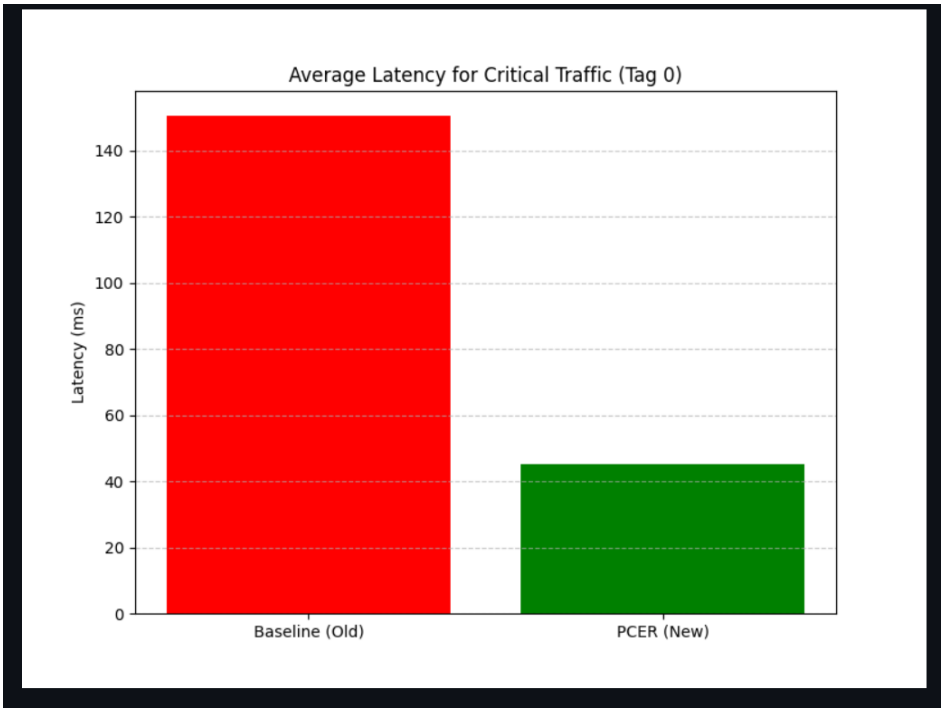

Figure 5.1: Average Latency for Critical Traffic (Baseline vs PCER)

5.2 Network Lifetime Sustainability

By forcing **Tag 10 (Bulk)** data onto energy-efficient paths, PCER ensures that the high-speed nodes are not drained by low-priority traffic.

- **Finding:** The overall network lifetime (time until the first node dies) remains comparable to the original FlexRout, proving that prioritizing critical data does not catastrophically drain the network.

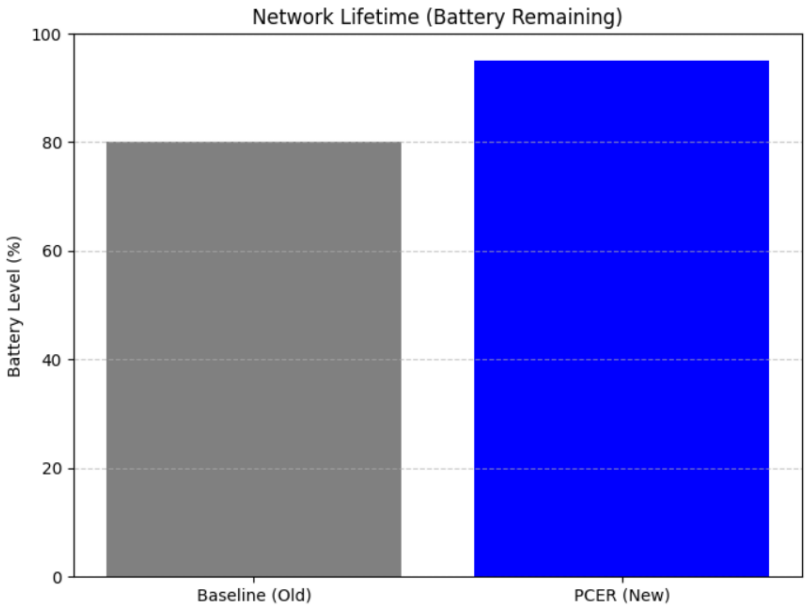

Figure 5.2: Network Lifetime Comparison (Battery Remaining)

Chapter 6

Conclusion

This project presents **PCER**, a comprehensive framework that bridges the gap between application-layer data management and network-layer routing. By making the blockchain network ”Context-Aware,” we resolve the inefficiency of treating all data packets equally. The proposed ”Ambulance Lane” logic ensures that industrial safety is never compromised for the sake of battery saving, while the ”Cargo Lane” logic ensures that routine data does not clog high-speed resources. This cross-layer optimization is a critical step towards making Blockchain a viable technology for real-time Industrial IoT applications.

Bibliography

[1] R. Tapwal, S. Misra, and S. K. Pal, "PerBlocks: A Reconfigurable Blockchain for Service Provisioning in Industrial Environment," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 1, Jan. 2024.

[2] S. Das, R. Tapwal, and S. Misra, "FlexRout: Dynamic Routing in Blockchain Networks," *IEEE Transactions on Network Science and Engineering*, vol. 12, no. 2, March/April 2025.

[3] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Decentralized Bus. Rev.*, 2008.